السنة الجامعية: 2027/2026
المقياس: تحليلات البيانات الضخمة

تقرير TP05

بثّ ومعالجة البيانات في الزمن الحقيقي باستخدام Apache Kafka

الطالبة: لينا زواري أحمد
السنة الثانية ماستر – ذكاء اصطناعي وعلوم البيانات

1. الهدف

محاكاة حسّاسات درجة الحرارة والرطوبة في الحرم الجامعي، وبثّ أحداثها بصيغة JSON عبر Apache Kafka (نمط KRaft وDocker Compose)، وبناء منتِج (Producer) ومستهلِك (Consumer) بلغة Python لاكتشاف درجات الحرارة المرتفعة (≥ 35 °م).

المكوّن	التفاصيل
Kafka	apache/kafka:3.9.1، عقدة واحدة، نمط KRaft (بدون ZooKeeper)
مكتبة Python	kafka-python==2.0.2 (داخل بيئة افتراضية)
الـ Topic / المجموعة	campus-sensors (3 أقسام، معامل نسخ 1) / tp05-campus-monitor

2. المهمة 1 – نشر Kafka

يشغّل الملف compose.yaml حاوية واحدة تعمل كـ broker وcontroller معًا على المنفذ localhost:9092، مع 3 أقسام افتراضية ومعاملات نسخ تساوي 1 ووحدة تخزين دائمة.

```
docker compose up -d
docker compose ps
docker compose logs --tail=50 kafka
```

حالة Docker Compose (مخرجات متوقعة)

```
(.venv) PS C:\Users\LAPTA\Downloads\tp05-kafka\tp05-kafka> docker compose up -d
[+] up 1/1
 ✓ Container tp05-kafka Started
(.venv) PS C:\Users\LAPTA\Downloads\tp05-kafka\tp05-kafka> docker compose ps
NAME        IMAGE              COMMAND                  SERVICE   CREATED       STATUS         PORTS
tp05-kafka  apache/kafka:3.9.1  "/__cacert_entrypoin…"   kafka     30 hours ago  Up 41 seconds  0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
(.venv) PS C:\Users\LAPTA\Downloads\tp05-kafka\tp05-kafka> docker compose logs --tail=50 kafka
tp05-kafka  | [2026-11-11 18:28:26,138] INFO [GroupMetadataManager brokerId=1] Finished loading offsets and group metadata from __consumer_offsets-26 in 13 millisecond
epoch 4, of which 13 milliseconds was spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
tp05-kafka  | [2026-11-11 18:28:26,138] INFO [GroupMetadataManager brokerId=1] Finished loading offsets and group metadata from __consumer_offsets-5 in 13 milliseconds
```

3. المهمة 2 – تثبيت مكتبة Python

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

4. المهمة 5 – فحص الـ Topic

أُنشئ الـ topic يدويًا قبل تشغيل البرنامجين (يظهر الاسم "campussensors" في نص التمرين بسبب انقسام السطر في ملف PDF، والاسم الصحيح هو campus-sensors):

```
docker exec -it tp05-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 \
  --create --if-not-exists --topic campus-sensors --partitions 3 --replication-
factor 1
docker exec -it tp05-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092 \
  --describe --topic campus-sensors
docker exec -it tp05-kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-
server localhost:9092 --list
```

وصف الـ Topic

```
Topic: campus-sensors   PartitionCount: 3   ReplicationFactor: 1
  Partition: 0   Leader: 1   Replicas: 1   Isr: 1
  Partition: 1   Leader: 1   Replicas: 1   Isr: 1
  Partition: 2   Leader: 1   Replicas: 1   Isr: 1
```

- **الأقسام (3):** ثلاثة سجلات مرتّبة ومستقلة، لكل منها offsets خاصة بها؛ وتسمح بما يصل إلى 3 مستهلكين متوازيين في مجموعة واحدة.
- **معامل النسخ (1):** نسخة واحدة من كل قسم على الـ broker الوحيد، فلا يوجد أي تكرار احتياطي.

5. المهمة 3 – المنتِج (Producer)

يرسل producer.py 30 حدثًا بصيغة JSON (حدث كل ثانية) تحتوي على event_id وsensor_id وtemperature_c (من 18 إلى 42) وhumidity_pct (من 25 إلى 85) وtimestamp. الإعداد "acks="all ينتظر تأكيد الـ broker، والدالة flush() ترسل الرسائل المخزّنة مؤقتًا قبل الإغلاق.

```
python producer.py
```

مخرجات المنتِج

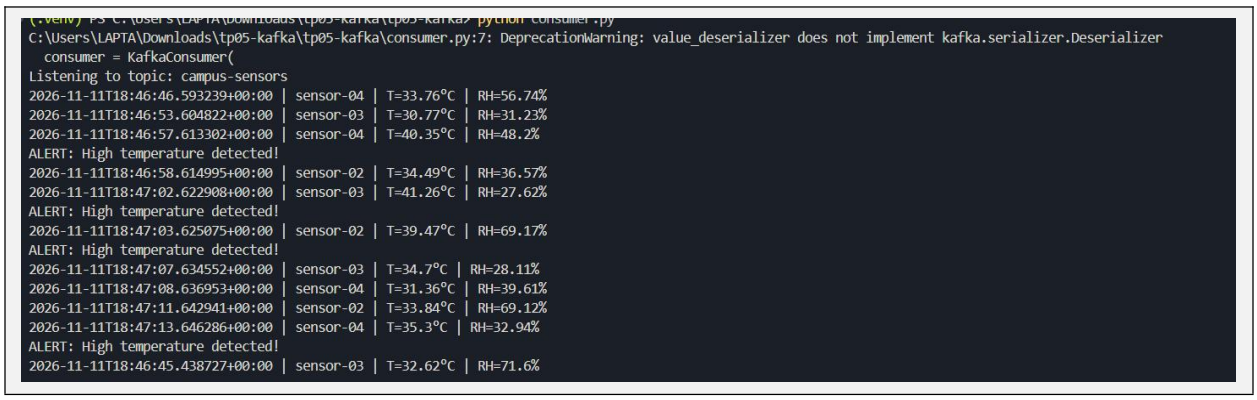
```
(.venv) PS C:\Users\LAPTA\Downloads\tp05-kafka\tp05-kafka> python producer.py
Sent: {'event_id': 1, 'sensor_id': 'sensor-03', 'temperature_c': 32.62, 'humidity_pct': 71.6, 'timestamp': '2026-11-11T18:46:45.438727+00:00'}
Sent: {'event_id': 2, 'sensor_id': 'sensor-04', 'temperature_c': 33.76, 'humidity_pct': 56.74, 'timestamp': '2026-11-11T18:46:46.593239+00:00'}
Sent: {'event_id': 3, 'sensor_id': 'sensor-01', 'temperature_c': 41.95, 'humidity_pct': 50.46, 'timestamp': '2026-11-11T18:46:47.594889+00:00'}
Sent: {'event_id': 4, 'sensor_id': 'sensor-04', 'temperature_c': 31.52, 'humidity_pct': 71.69, 'timestamp': '2026-11-11T18:46:48.596779+00:00'}
Sent: {'event_id': 5, 'sensor_id': 'sensor-02', 'temperature_c': 23.62, 'humidity_pct': 35.96, 'timestamp': '2026-11-11T18:46:49.598222+00:00'}
Sent: {'event_id': 6, 'sensor_id': 'sensor-01', 'temperature_c': 38.75, 'humidity_pct': 70.12, 'timestamp': '2026-11-11T18:46:50.599765+00:00'}
Sent: {'event_id': 7, 'sensor_id': 'sensor-04', 'temperature_c': 26.3, 'humidity_pct': 75.85, 'timestamp': '2026-11-11T18:46:51.601052+00:00'}
Sent: {'event_id': 8, 'sensor_id': 'sensor-03', 'temperature_c': 36.45, 'humidity_pct': 33.45, 'timestamp': '2026-11-11T18:46:52.602490+00:00'}
Sent: {'event_id': 9, 'sensor_id': 'sensor-03', 'temperature_c': 30.77, 'humidity_pct': 31.23, 'timestamp': '2026-11-11T18:46:53.604822+00:00'}
Sent: {'event_id': 10, 'sensor_id': 'sensor-03', 'temperature_c': 18.55, 'humidity_pct': 69.61, 'timestamp': '2026-11-11T18:46:54.608021+00:00'}
Sent: {'event_id': 11, 'sensor_id': 'sensor-03', 'temperature_c': 25.1, 'humidity_pct': 46.77, 'timestamp': '2026-11-11T18:46:55.609532+00:00'}
Sent: {'event_id': 12, 'sensor_id': 'sensor-02', 'temperature_c': 38.7, 'humidity_pct': 29.97, 'timestamp': '2026-11-11T18:46:56.611198+00:00'}
Sent: {'event_id': 13, 'sensor_id': 'sensor-04', 'temperature_c': 40.35, 'humidity_pct': 48.2, 'timestamp': '2026-11-11T18:46:57.613302+00:00'}
```

6. المهمة 4 – المستهلِك (Consumer)

يقرأ consumer.py (المجموعة tp05-campus-monitor) الـ topic ويفك ترميز JSON ويطبع كل قياس، ويطبع تنبيهًا عندما تكون الحرارة ≥ 35 °م (حوالي 29% من القراءات، لأن (42 − 35) / (42 − 18) ≈ 0.29). يُشغَّل المستهلِك أولًا ثم المنتِج.

```
python consumer.py
```

مخرجات المستهلِك



7. مهام للتعمّق

المهمة	النتيجة
100 حدث	السلوك نفسه مع مدة أطول؛ يصل 100 رسالة وتزداد التنبيهات بشكل متناسب.
المتوسط المتحرك	consumer_avg.py: يتقارب إلى حوالي 30 °م.
مستهلِك ثانٍ بـ group_id مختلف	تستقبل كل مجموعة جميع الرسائل (نمط النشر والاشتراك).
مستهلِك ثانٍ بنفس group_id	تُقسَّم الأقسام بينهما؛ كل رسالة تصل إلى مستهلِك واحد فقط (توزيع الحمل).
قسم واحد مقابل 3 أقسام	يتغير: التوازي ونطاق ضمان الترتيب. لا يتغير: broker واحد، لا نسخ، محتوى الرسائل.
حقل campus_building والتصفية / CSV	يضيف producer_extended.py الحقل؛ ويصفّي consumer_csv.py مبنى واحدًا ويكتب الملف events.csv.

8. إجابات الأسئلة

1. ما المشكلة التي يحلّها Kafka؟

يفصل بين المنتِجين والمستهلِكين: تُحفظ الأحداث في سجل دائم ومرتّب وقابل للتوسع، وتستطيع تطبيقات عديدة قراءتها في الزمن الحقيقي، بدل استخدام ملفات ثابتة.

2. التعريفات

Producer (المنتِج)	تطبيق ينشر الأحداث في topic.
Consumer (المستهلِك)	تطبيق يقرأ الأحداث من topic.
Broker (الوسيط)	خادم Kafka الذي يخزّن البيانات ويخدم العملاء.
Topic (الموضوع)	قناة مسمّاة تجمع أحداثًا من النوع نفسه.
Partition (القسم)	سجل مرتّب بالإضافة فقط؛ جزء من الـ topic.
Offset (الإزاحة)	الرقم التسلسلي للرسالة داخل القسم.
Consumer group (مجموعة المستهلكين)	مستهلكون لهم نفس group_id يتقاسمون أقسام الـ topic.

3. ما الفرق بين الـ topic والـ partition؟

الـ topic قناة منطقية، والـ partition تقسيمه الفعلي. الترتيب مضمون داخل القسم الواحد فقط.

4. لماذا 'auto_offset_reset='earliest؟ ومتى يُطبَّق؟

ليقرأ المستهلِك من بداية السجل. يُطبَّق فقط عندما لا يوجد offset محفوظ صالح للمجموعة (مجموعة جديدة أو offset منتهي)؛ وإلا يكمل من آخر offset محفوظ.

5. ما غرض group ID؟ وماذا لو استُخدم ID جديد؟

يحدد المجموعة ويُحفظ باسمها الـ offset، ويتقاسم أعضاؤها الأقسام. ID جديد يعني مجموعة جديدة بلا offsets تقرأ كل الرسائل بشكل مستقل (من البداية مع 'earliest').

6. لماذا JSON مناسب؟ وما حدّه؟

مقروء وبسيط ولا يحتاج schema. الحد: حجمه كبير ولا يفرض schema أو أنواعًا صارمة (يُفضَّل Avro/Protobuf في الإنتاج).

7. كيف يبسّط Docker Compose النشر؟

ملف واحد يصف الصورة والمنافذ والمتغيرات والـ volumes، فتُشغَّل بيئة قابلة للتكرار بأمر واحد بدل التثبيت والضبط اليدوي.

8. حدود إعداد الـ broker الواحد

- لا تحمّل للأعطال ولا نسخ احتياطية (معامل النسخ 1).
- الأقسام لا تعطي توازيًا حقيقيًا كبيرًا على عقدة واحدة.
- بلا أمان (PLAINTEXT بدون TLS/SASL/ACL) وبلا مراقبة.
- يمكن الوصول إليه من localhost فقط.

```
(.venv) PS C:\Users\LAPTA\Downloads\tp05-kafka\tp05-kafka>    Test-NetConnection localhost -Port 9092


ComputerName     : localhost
RemoteAddress    : ::1
RemotePort       : 9092
InterfaceAlias   : Loopback Pseudo-Interface 1
SourceAddress    : ::1
TcpTestSucceeded : True
```

9. المشاكل والحلول

المشكلة	الحل
NoBrokersAvailable	تحقق من docker compose ps وانتظر بضع ثوانٍ بعد التشغيل.
المنفذ 9092 مشغول	غيّر المنفذ على جهة المضيف وحدّث bootstrap_servers والـ advertised listener.
خطأ kafka.vendor.six.moves (مع Python 3.12+)	استخدم kafka-python>=2.1 أو kafka-python-ng.
المستهلك لا يعرض الرسائل القديمة	للمجموعة offsets محفوظة؛ استخدم group_id جديدًا.

10. الخلاصة والتنظيف

نُشر Kafka بنمط KRaft، وبُثّت أحداث الحسّاسات وحُلّلت في الزمن الحقيقي بمنتِج ومستهلِك Python. هذا الإعداد بـ broker واحد مناسب للتعلّم وليس للإنتاج.

```
docker compose down        # stop
docker compose down -v     # also delete Kafka data
```